

ISTEC Porto
Instituto Superior de Tecnologias Avançadas do Porto
Licenciatura em Engenharia Informática
Ano Letivo: 2025/2026

Unidade Curricular: Programação II
Relatorio Projeto Prático de Programação II

Autor:

Gonçalo Silva Pereira
goncalo.pereira.2022051@my.istec.pt

Estudante do 1º ano da Licenciatura de Engenharia Informática

Docente: João Rebelo

Porto, 25 junho de 2026

Índice

Introdução.....	3
Descrição do Problema.....	3
Enumeração dos Objetivos.....	4
Funcionalidades do Sistema.....	4
Portal do Cliente.....	4
Portal do Técnico.....	5
Estado da Arte.....	5
Metodologia.....	6
Desenvolvimento, Apuramento de Resultados e Modelo de Classes.....	7
Módulos Funcionais e Rotinas de Validação.....	7
Abordagem do Modelo de Classes e Relações OO.....	7
Futuras Melhorias.....	9
Considerações Finais.....	10
Anexos.....	11
Anexo A – Repositório Digital do Projeto (GitHub).....	11
Anexo B – Artefactos de Modelação UML.....	11

Introducao

O presente documento constitui o relatório final de desenvolvimento do projeto prático realizado no âmbito da unidade curricular de Programação II. O projeto consistiu no desenvolvimento do “**DeskFlow**”, um software desktop de HelpDesk e suporte técnico concebido para otimizar a gestão de incidentes e a comunicação com os clientes.

A aplicação foi integralmente desenvolvida em C#, recorrendo à tecnologia **Windows Presentation Foundation (WPF)** com XAML para a criação de uma interface gráfica interativa, moderna e reativa. A lógica do sistema assentou rigorosamente nos pilares da **Programação Orientada a Objetos (POO)**.

Relativamente à persistência de dados, e face aos reajustes metodológicos ao longo da unidade curricular, a solução foi estruturada para operar inteiramente através de coleções genéricas e reativas (*ObservableCollection*) em memória RAM. Esta abordagem permitiu focar o desenvolvimento na manipulação avançada de objetos em tempo real, na aplicação de consultas LINQ e nos mecanismos nativos de *Data Binding* do WPF, garantindo uma execução fluida e a total integridade dos dados durante o tempo de execução da aplicação.

Descricao do Problema

As empresas modernas enfrentam grandes dificuldades em gerir os pedidos de assistência, dúvidas e incidentes reportados pelos seus clientes. A ausência de um sistema estruturado e centralizado de HelpDesk resulta em três falhas críticas no ecossistema de suporte:

- **Perda de rastreabilidade:** Quando os pedidos são feitos por canais informais (como e-mail ou telefone), perdem-se facilmente. Sem um estado definido, torna-se impossível controlar quais os problemas que já foram resolvidos e quais estão pendentes na fila de suporte;
- **Falta de priorização:** Sem uma triagem e categorização visual clara, incidentes altamente críticos (como a falha total de um servidor corporativo) correm o risco de ser tratados com a mesma urgência que dúvidas simples de utilização, prejudicando severamente os Acordos de Nível de Serviço (SLA);
- **Inexistência de histórico centralizado:** A dificuldade em associar um histórico de incidentes a um cliente específico impede que a equipa técnica identifique problemas recorrentes na infraestrutura ou nos produtos do utilizador.

O **DeskFlow** foi desenvolvido especificamente para combater estas lacunas. Através da centralização de dados em coleções na memória RAM da aplicação, o sistema garante o mapeamento imediato de cada ocorrência a uma entidade concreta, distribuindo o fluxo de trabalho de forma lógica e auditável entre clientes e técnicos.

Enumeraçao dos Objetivos

O objetivo central deste projeto consistiu em consolidar de forma prática os conteúdos programáticos da unidade curricular de Programação II, desenvolvendo uma solução comercialmente realista, robusta e totalmente funcional em ambiente desktop.

Os objetivos específicos alcançados incluem:

- **Modelação de Dados em POO:** Desenho e implementação de uma estrutura de classes limpa, aplicando encapsulamento estrito (atributos privados e propriedades públicas com métodos *getters/setters*) e construtores bem definidos;
- **Implementação de Relações UML:** Aplicação prática de relações estruturais fundamentais entre as entidades do sistema, nomeadamente Herança (classe abstrata *Person* estendida para *Customer* e *Technician*), Associação e Composição;
- **Persistência em Memória e Helpers de Dados:** Manipulação centralizada dos dados da aplicação através de coleções reativas (*ObservableCollection*) instanciadas globalmente no arranque do sistema, delegando as operações de CRUD (criar utilizadores, abrir, atribuir e fechar tickets) para classes utilitárias isoladas (*Helpers*), garantindo a integridade dos dados através de consultas LINQ;
- **Desenvolvimento de UI Reativa em WPF:** Criação de uma interface gráfica amigável e moderna, recorrendo a controlos avançados (como *DataGrid* com estilos personalizados e ordenação, *ListBox* estruturada para histórico de intervenções e *comboboxes*) , totalmente amarrada aos dados (*Data Binding*) e com tratamento de exceções robusto (*try-catch*) para evitar falhas inesperadas.

Funcionalidades do Sistema

De forma a cumprir os objetivos delineados, o **DeskFlow** foi dotado de um conjunto completo de funcionalidades que cobrem os requisitos de um ecossistema de suporte real, dividido em dois portais autónomos baseados no perfil do utilizador logado:

Portal do Cliente

- **Abertura de Tickets:** Criação de novos tickets de suporte, permitindo ao cliente definir o título, a descrição detalhada do problema e o nível de prioridade (Alta, Média, Baixa);
- **Histórico Pessoal de Tickets:** Visualização de uma listagem em tempo real contendo apenas os tickets abertos pelo próprio cliente, facilitando o acompanhamento dos estados;
- **Interação via Intervenções:** Acesso a uma área de chat/detalhes onde o cliente pode enviar mensagens e ler as respostas inseridas pela equipa técnica;
- **Encerramento Autónomo:** Possibilidade de o cliente fechar o seu próprio ticket caso o problema seja resolvido antes da intervenção técnica;

- **Gestão de Perfil:** Consulta e atualização dos dados pessoais do cliente.

Portal do Técnico

- **Triagem e Fila de Espera Global:** Acesso a uma tabela centralizada que lista todos os tickets do sistema que ainda não têm um técnico associado;
- **Captura de Ocorrências (Assignment):** Através de um duplo clique na fila de espera, o técnico pode associar o ticket a si próprio;
- **Painel de Trabalho ("Os Meus Tickets"):** Monitorização isolada dos chamados que estão sob a responsabilidade direta do técnico logado;
- **Inserção de Intervenções:** Possibilidade de registar notas de suporte técnico e respostas textuais no histórico do ticket;
- **Encerramento Técnico:** Resolução definitiva e fecho de tickets concluídos.
- **Gestão de Perfil Profissional:** Edição de dados e da área de especialização técnica.

Estado da Arte

No mercado atual de TI, existem ferramentas de HelpDesk e Service Desk de grande escala consolidadas, tais como o *Zendesk*, o *Freshdesk* ou o *Jira Service Desk*. Embora estas plataformas ofereçam ecossistemas robustos e completos, elas apresentam barreiras significativas para pequenas equipas ou cenários de suporte interno: exigem subscrições dispendiosas baseadas na nuvem, dependem de ligações constantes à internet, possuem interfaces complexas e requerem formação avançada e demorada para os utilizadores.

Por outro lado, muitas pequenas organizações recorrem a soluções rudimentares baseadas em registos em papel ou folhas de cálculo partilhadas (como o *Microsoft Excel*). Estas abordagens falham gravemente pela total ausência de integridade relacional, falta de controlo de acessos concorrentes e incapacidade de gerar notificações ou fluxos de trabalho automáticos.

O **DeskFlow** posiciona-se precisamente no equilíbrio entre estes dois extremos. Como uma aplicação nativa para ambiente desktop, elimina a dependência de subscrições em infraestruturas *cloud* e foca-se na rapidez de resposta do operador técnico. Ao simplificar o fluxo de triagem através de uma interface reativa em WPF e centralizar a lógica de dados diretamente na memória RAM, o sistema mitiga o risco de erro humano associado às folhas de cálculo, sem sobrecarregar a organização com funcionalidades redundantes ou configurações burocráticas.

Metodologia

O desenvolvimento do **DeskFlow** seguiu uma metodologia de arquitetura em camadas autónomas, garantindo a separação de conceitos (*Separation of Concerns*), o que facilita a manutenção do código e a escalabilidade da aplicação. A solução está estruturada em três camadas fundamentais:

- **Camada de Apresentação (WPF / XAML):** Responsável exclusivamente pela interface visual com o utilizador e pela captura de eventos (como cliques em botões ou duplos cliques nas tabelas). Esta camada utiliza os mecanismos nativos de *Data Binding* do WPF, permitindo que a interface reaja e se atualize automaticamente sempre que os dados subjacentes sofram alterações.
- **Camada de Lógica de Negócio (POO em C#):** A camada central onde residem os modelos de dados e as regras de negócio do HelpDesk. É aqui que se processam as validações de permissões, a alteração de estados dos tickets, as regras de controlo de escrita nas intervenções e os comportamentos de herança e polimorfismo das entidades.
- **Camada de Gestão de Dados em Memória (Helpers e Coleções Globais):** Em substituição da persistência clássica em bases de dados externas, foi implementada uma camada de gestão de estado centralizada no ciclo de vida da aplicação. Os dados são armazenados em coleções globais dinâmicas (*ObservableCollection*) instanciadas no ficheiro *App.xaml.cs*. A manipulação, filtragem e atualização destas coleções são delegadas a classes utilitárias isoladas (*Helpers*, como o *TicketHelper*), que utilizam expressões LINQ para garantir a integridade relacional entre os objetos (ex: associar corretamente um técnico ou cliente a um ticket pelo seu identificador único), simulando com eficácia o comportamento de um repositório de dados.

Desenvolvimento, Apuramento de Resultados e Modelo de Classes

O desenvolvimento do **DeskFlow** foi dividido em módulos funcionais que se traduzem na manipulação de objetos e na aplicação rigorosa dos conceitos de Programação Orientada a Objetos (POO).

Módulos Funcionais e Rotinas de Validação

- **Módulo de Gestão de Utilizadores:** Implementação do fluxo de autenticação e diferenciação de permissões. As entidades são carregadas a partir da coleção global `IstUsers`, garantindo a validação de dados de entrada (como campos vazios) no ecrã de Login e nos controlos de Perfil;
- **Módulo de Gestão de Tickets e Intervenções:** Mecanismo centralizado que rege a criação, captura (atribuição) e encerramento de ocorrências. As intervenções funcionam como um histórico incremental associado a cada ticket, com controlo de acessos que impede técnicos não associados de interagir com o chamado;
- **Rotinas de Detecção de Erros:** Utilização estruturada de blocos try-catch em operações críticas da interface gráfica e na conversão de tipos (*casting*), garantindo o tratamento de exceções e assegurando que a aplicação não crashe inesperadamente diante de dados incompatíveis;
- **Apuramento de Resultados (Dashboard):** A verificação da eficácia e consistência do sistema é realizada através da extração de métricas em tempo real diretamente das coleções em memória, monitorizando indicadores como o total de tickets abertos vs. resolvidos e a distribuição de chamados por técnico.

Abordagem do Modelo de Classes e Relações OO

Para cumprir os requisitos académicos de herança e modelação de dados, a arquitetura do sistema foi desenhada com base em três tipos de relações UML fundamentais:

Relação de Herança (Especialização):

Definição da classe abstrata `Person`, que serve de base e concentra os atributos comuns (como `Name`, `Email`, `Password`). As classes `Customer` (que adiciona `CompanyName`) e `Technician` (que adiciona `Specialization`) herdam diretamente de `Person`, demonstrando o conceito clássico de especialização e reaproveitamento de código.

Relação de Associação:

O modelo assenta em associações simples mas estritas entre os objetos. A classe `Ticket` possui uma associação direta com a classe `Customer` (identificando o autor da

abertura) e com a classe Technician (identificando o operador responsável pelo caso). Estas associações permitem interligar os objetos em memória de forma limpa, mantendo a rastreabilidade total do fluxo.

Relação de Composição / Agregação:

A classe Ticket gere uma coleção de objetos da classe Intervention. Na lógica de negócio do sistema, as intervenções técnicas e mensagens de chat estão intrinsecamente ligadas ao ciclo de vida da ocorrência; se um ticket for removido do sistema, o seu histórico de notas e intervenções perde o propósito, sendo gerido em cascata.

Mecanismo de Polimorfismo (Destaque Técnico):

Como refinamento da arquitetura, a propriedade ClosedBy dentro da classe Ticket foi tipada diretamente com a classe abstrata mãe Person. Isto permite aplicar o conceito de polimorfismo em tempo de execução: o atributo é capaz de receber e armazenar tanto um objeto do tipo Customer como do tipo Technician (uma vez que ambos são uma Person), refletindo fielmente a regra de negócio em que o encerramento do ticket pode ser efetuado de forma autónoma por qualquer uma das partes envolvidas.

Futuras Melhorias

Embora o **DeskFlow** cumpra com rigor todos os requisitos pedagógicos e funcionais estabelecidos para a unidade curricular, foram identificadas diversas áreas que poderiam ser otimizadas e expandidas numa versão futura, visando a transição do sistema para um ambiente de produção real:

- **Migração para uma Base de Dados Relacional:** A utilização de coleções em memória RAM (ObservableCollection) foi ideal para consolidar a lógica de POO, mas não é viável para um cenário real de larga escala. Uma melhoria prioritária seria a integração planeada com o SQL Server, garantindo a persistência permanente e segura dos dados mesmo após o encerramento da aplicação;
- **Robustez, Validações e Tratamento de Erros:** Há margem para fortificar o código com uma cobertura mais densa de blocos try-catch. Seria importante implementar rotinas de validação complexas (especialmente no ecrã de autenticação e no registo de emails) com mensagens de retorno de erro (*Error Providers*) mais claras e informativas para o utilizador;
- **Refinamento, Coerência e Responsividade da UI:** Apesar de a interface gráfica em WPF/XAML se apresentar limpa e funcional, necessita de melhorias na responsividade para diferentes resoluções de ecrã, além de uma revisão estética detalhada para corrigir pequenas desorganizações e garantir uma coerência visual absoluta entre todas as janelas e controlos;
- **Complexidade no Fluxo de Tickets:** Expandir o módulo de ocorrências, permitindo a categorização de problemas por subáreas, prazos de resolução automáticos (SLA) e anexação de ficheiros ou capturas de ecrã;
- **Módulo Administrativo (Back Office) e Níveis de Técnicos:** Implementar uma área de administração global para gerir a plataforma (criar, editar e remover técnicos e clientes). Adicionalmente, seria útil criar diferentes níveis de técnicos (Ex: Nível 1 para suporte geral, Nível 2 para infraestrutura), permitindo que a fila de espera fizesse uma triagem automática com base na especialidade de cada operador.

Consideracoes Finais

O desenvolvimento do **DeskFlow** constituiu uma experiência acadêmica enriquecedora e desafiante, assumindo-se como o verdadeiro ponto de convergência de toda a matéria lecionada e absorvida ao longo da unidade curricular de Programação II. Mais do que a simples criação de um software funcional, este projeto individual funcionou como um agregador holístico de conceitos abstratos, forçando a sua tradução em linhas de código coesas, estruturadas e comercialmente realistas.

Durante o semestre, os pilares da Programação Orientada a Objetos foram assimilados de forma teórica; no entanto, foi o desenho desta solução que solidificou a mecânica do encapsulamento estrito (com atributos privados e propriedades públicas) , o ciclo de vida dos objetos através de construtores e, crucialmente, o reaproveitamento de código via herança — visível na especialização da classe abstrata *Person* nas entidades *Customer* e *Technician*. Adicionalmente, a implementação das relações estruturais (como a associação entre tickets e utilizadores e a composição no histórico de intervenções) permitiu compreender como mapear em memória a complexidade de um problema do mundo real, garantindo a integridade relacional sem o auxílio inicial de um motor de base de dados.

A necessária adaptação metodológica para uma arquitetura baseada inteiramente em memória RAM, longe de mitigar o valor do projeto, acabou por expandir o leque de competências adquiridas. Esta mudança exigiu um domínio aprofundado sobre coleções dinâmicas e reativas (*ObservableCollection*), a manipulação de dados por referência e a utilização avançada de expressões LINQ para filtragem e pesquisa de objetos em tempo real — replicando a lógica de repositórios através de *Helpers* isolados. Da mesma forma, o polimorfismo dinâmico deixou de ser um conceito puramente conceptual e passou a ser a solução elegante para a propriedade *ClosedBy*, permitindo ao sistema tratar de forma uniforme o encerramento de um chamado, quer este fosse acionado por um cliente ou por um técnico.

Complementarmente, a integração da lógica de negócio com a interface gráfica em Windows Presentation Foundation (WPF) e XAML materializou a importância do *Data Binding* e do tratamento rigoroso de eventos na criação de experiências reativas para o utilizador. O desenvolvimento do *DeskFlow* provou que a robustez de um software depende tanto da elegância do seu modelo de classes como da eficácia das suas rotinas de validação e blocos de deteção de erros (try-catch).

Em suma, os objetivos estipulados na proposta inicial foram plenamente alcançados. O **DeskFlow** resultou numa aplicação desktop ágil, intuitiva e modular que não só resolve de forma eficaz os problemas de rastreabilidade e priorização propostos , como serve de

testemunho prático da maturidade e do conhecimento técnico consolidado ao longo de toda a disciplina de Programação II.

Anexos

Anexo A – Repositório Digital do Projeto (GitHub)

Todo o código-fonte da aplicação DeskFlow, bem como o histórico de *commits*, controlo de versões e documentação de suporte, encontra-se alojado publicamente na plataforma GitHub. O ficheiro principal README.md serve como guia de introdução e contém as credenciais de teste para execução imediata do ambiente em memória.

- **Ligação Direta para o Repositório:** <https://github.com/GoncaloPerei/DeskFlow>

Anexo B – Artefactos de Modelação UML

Dentro do repositório indicado no Anexo A, na pasta dedicada /docs, foram disponibilizados os diagramas técnicos em alta resolução que sustentam a arquitetura de Programação Orientada a Objetos da aplicação:

- **Diagrama de Classes (DC):** Mapeamento completo de todas as propriedades, métodos, encapsulamento estrito, relações de herança, associações e composição.
 - *Caminho no Repositório:* /Docs/Diagrams/Diagrama_Classes.draw.io
- **Diagrama de Casos de Utilização (Use Cases):** Representação visual das fronteiras do sistema, detalhando as permissões e interações do Cliente e do Técnico com os módulos do ecossistema.
 - *Caminho no Repositório:* /Docs/Diagrams/Diagrama_Use_Case.draw.io
- **Fluxograma Prototipado (Flowchart):** Mapeamento sequencial e lógico que descreve o fluxo de execução do sistema (processo de autenticação, triagem de perfil, captura dinâmica de ocorrências pelo técnico e o ciclo de vida de fecho do ticket), demonstrando a árvore de decisão implementada no código C#.
 - *Caminho no Repositório:* /Docs/Diagrams/Fluxograma.draw.io